

Real-time Sentiment Analysis YouTube Comments using Kafka and Pyspark

Kim-Oanh Nguyen-Trieu^{1,2}

Uyen-Nhi Nguyen-Phan^{1,2}
Trong-Hop Do^{1,2}

Tuan-Kiet Tran-Ngo^{1,2}

¹Faculty of Information Science and Engineering, University of Information Technology, Ho Chi Minh City, Vietnam

²Vietnam National University, Ho Chi Minh City, Vietnam

{20521729, 20521714, 20521505}@gm.uit.edu.vn, hopdt@uit.edu.vn

Abstract—This project introduces a scalable and efficient solution for real-time sentiment analysis on YouTube comments using a robust technological stack. Leveraging Docker for containerization, Kafka for real-time data streaming, and PySpark for distributed data processing, our system provides an end-to-end pipeline for sentiment analysis. YouTube comments are streamed through Kafka topics, processed in parallel using PySpark, and sentiment scores are computed in real-time. The modular and scalable Docker containers enable easy deployment and scaling of the entire system. Our approach enhances the ability to monitor and understand audience sentiments in real-time, providing valuable insights for content creators and stakeholders in the online media industry.

Index Terms—Big data, Sentiment analyst, Docker, Kafka, Pyspark, Youtube.

I. INTRODUCTION

In the age of information, online platforms like YouTube have become hubs of diverse user-generated content, fostering a global community that actively engages through comments and discussions. The sentiments expressed within these comments are rich sources of information, offering insights into user opinions, feedback, and emotional responses. Real-time sentiment analysis of such comments is paramount for understanding audience sentiment, enabling content creators, marketers, and researchers to adapt strategies, enhance user experiences, and make data-driven decisions.

This paper introduces a comprehensive solution for real-time sentiment analysis from YouTube comments, leveraging a synergistic combination of Docker, Apache Kafka, and PySpark. The integration of these technologies not only addresses the challenges posed by the dynamic and voluminous nature of social media data but also provides a scalable and efficient framework for extracting valuable insights in real-time.

The deployment of Docker containers facilitates a streamlined and portable deployment process, ensuring the system's consistency across different environments. Apache Kafka, as a robust distributed streaming platform, enables the seamless flow of YouTube comments for real-time processing, ensuring fault tolerance and scalability. PySpark, a powerful distributed data processing library, is employed for sentiment analysis, allowing the system to efficiently classify sentiments within the comments, even when dealing with large datasets.

We shall provide related works in Section II and the system model in Section III of the remaining portion of the paper. Data

and models are presented in Section IV. Then, in section V, we assess and analyze errors. Finally, in section VI, we conclude and suggest a course of action for future development.

II. RELATED WORK

Sentiment analysis serves as a pivotal tool for deciphering user sentiment during interactions with content on the platform [1]. Its application extends across various sectors including businesses, research endeavors, and organizational initiatives, offering valuable insights into community feedback and evaluations about specific content. The real-time implementation of sentiment analysis is imperative for swiftly responding to emerging events or trends on YouTube. This agility enables proactive adjustments to user community sentiment, ensuring timely adaptations to evolving preferences and perceptions.

The integration of Docker into the deployment process plays a crucial role in ensuring the seamless implementation of sentiment analysis. Docker simplifies deployment procedures while maintaining consistency between development and deployment environments [2]. Its containerization technology facilitates the creation of packaged and portable environments, facilitating smooth transitions across different systems.

In the architecture of this project, Kafka emerges as a cornerstone event-streaming platform, facilitating the efficient management and streaming of event data across various system components [3]. Leveraging Kafka enhances the system's ability to connect and transmit data flexibly and at scale, ensuring seamless communication between different modules.

PySpark, serving as the Python interface for Apache Spark, emerges as a potent tool for data processing and analysis within this context [4]. By leveraging PySpark, the system efficiently handles large volumes of data extracted from YouTube comments, harnessing the immense processing power of Apache Spark. This integration empowers the system to conduct comprehensive analyses, extracting meaningful insights from the vast troves of user-generated content on the platform.

III. SYSTEM ARCHITECTURE

The Real-time Sentiment Analysis system for YouTube comments leverages Kafka as a scalable and distributed streaming platform, and PySpark for efficient and parallelized processing of data. This architecture allows for the continuous

and real-time analysis of sentiment in user comments on YouTube videos.

A. YouTube API

Within our system architecture, the YouTube API serves as a critical component facilitating real-time interaction with YouTube's vast repository of video content. Leveraging this API, our system seamlessly fetches comments directly from YouTube videos. Acting as the primary data source for our streaming pipeline, the YouTube API enables us to continuously access and retrieve the latest comments, ensuring that our analyses are based on up-to-date user engagement and feedback.

B. Kafka

At the heart of our architecture lies Kafka, a robust messaging broker and streaming platform renowned for its scalability and fault tolerance. Serving as the central nervous system of our data ingestion process, Kafka efficiently handles the seamless flow of YouTube comments in real-time. Comments are meticulously published as messages to designated Kafka topics, guaranteeing their durability and reliability throughout the streaming pipeline. Leveraging Kafka Streams, our system effortlessly processes incoming comments, executing critical tasks such as data normalization, filtering, and transformation with unparalleled efficiency.

C. Comment Ingestion Service

Dedicated to the seamless integration of YouTube comments into our streaming platform, the Comment Ingestion Service plays a pivotal role in ensuring the timely and reliable ingestion of user-generated content. Tasked with fetching comments directly from the YouTube API, this service meticulously orchestrates the transfer of data, efficiently publishing them to their respective Kafka topics. By streamlining the ingestion process, this service optimizes the overall efficiency and reliability of our streaming platform, guaranteeing the uninterrupted flow of user engagement data for downstream analysis.

D. Sentiment Analysis Module (PySpark)

Harnessing the power of PySpark, our sentiment analysis module empowers our system with the ability to analyze the emotional tone and sentiment conveyed within YouTube comments. PySpark's inherent capability to distribute computations across a cluster enables us to perform sentiment analysis at scale, seamlessly processing vast volumes of user-generated content in real-time. Whether utilizing pre-trained machine learning models or employing a rule-based approach, our sentiment analysis module leverages PySpark's parallel processing capabilities to enhance the efficiency and accuracy of sentiment classification, providing invaluable insights into user sentiment and engagement trends.

IV. DATASET AND MODEL

A. Dataset

The Amazon Fine Food Reviews dataset reviews exquisite meals, consisting of 568,454 reviews on high-quality food products from Amazon. The includes all reviews up to October 2012 and spans more than ten years. Reviews from all other Amazon categories and also included. This dataset includes:

- **ProductId:** A unique identifier for each product.
- **UserId:** The unique identifier for each user providing the review.
- **ProfileName:** The profile name of the user who wrote the review.
- **HelpfulnessNumerator:** The numerator of the helpfulness rating given to the review by other users.
- **HelpfulnessDenominator:** The denominator of the helpfulness rating given to the review by other users.
- **Score:** The rating score given by the user (1 to 5).
- **Time:** The timestamp when the review was posted.
- **Summary:** A summary or title of the review.
- **Text:** The detailed text content of the review.

1) *Preparing data:* In this work, we assign a rating - positive reviews as 2, negative reviews as 0, and neutral reviews as 1.

2) Text Pre-processing:

a) *Creating Final Dataset with Apt Columns:* To ensure that the text data is properly prepared for modeling, several pre-processing steps are implemented through User Defined Functions (UDFs). These steps include:

- **Convert to Lower Case:** This step ensures uniformity in the text data by converting all characters to lowercase, preventing the model from treating words with different cases as different entities.
- **Remove Punctuations and Alphanumeric Words:** Punctuations and alphanumeric characters do not usually contribute to the semantics of the text and can add noise to the data. Hence, they are removed to focus on meaningful words.
- **Remove Stopwords:** Stopwords are common words (e.g., "the", "is", and "at") that occur frequently in the language but carry little semantic meaning. Removing them helps reduce the data's dimensionality and improve model efficiency.

b) *Part-of-Speech (POS) Tagging:* POS tagging is performed to assign a grammatical category (such as noun, verb, adjective, etc.) to each word in the text. This information is valuable for various Natural Language Processing (NLP) tasks, including lemmatization.

c) *Lemmatization:* Lemmatization involves reducing words to their base or root form, which helps standardize words with similar meanings. This step ensures that variations of a word are treated as the same entity during analysis, thereby improving the accuracy of the model.

3) Preparing Data For Modeling:

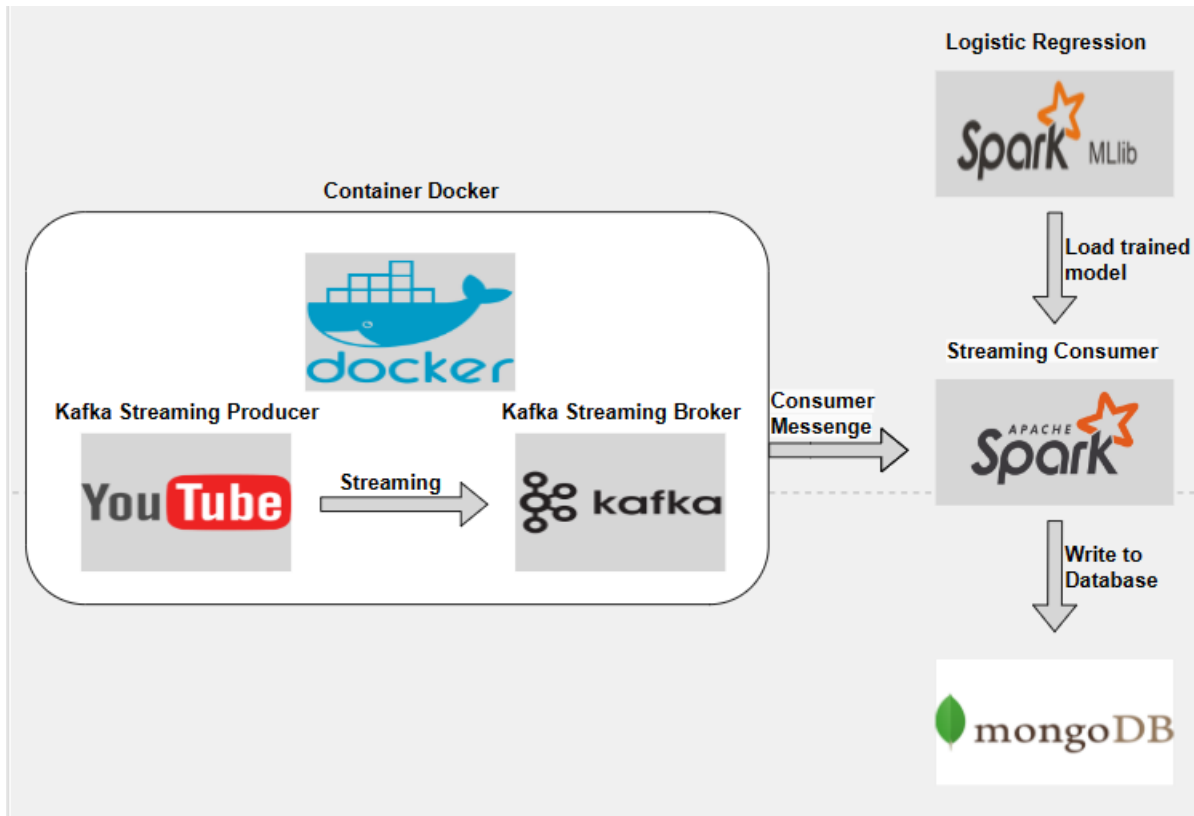


Fig. 1. System Architecture.

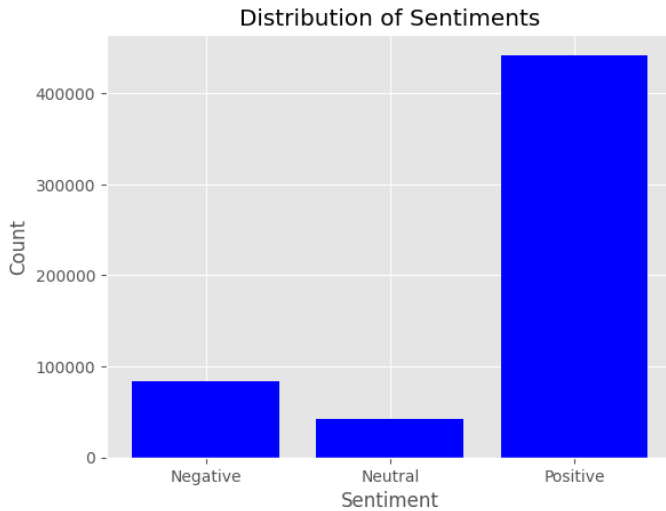


Fig. 2. Distribution of Label.

a) Creating Final Dataset with Apt Columns: After pre-processing, the text data is structured into a final dataset with appropriate columns, ensuring that it is ready for modeling. This dataset typically includes features extracted from the text along with the target variable (if applicable).

b) Dividing it into Training and Test Set: The dataset is split into two subsets: the training set and the test set. The

training set is used to train the model, while the test set is used to evaluate its performance. This ensures an unbiased assessment of the model's generalization ability.

c) Tokenizing the Training Set and Creating TF-IDF Matrix using HashingTF:

- **Tokenization:** The text data is tokenized using a tokenizer, breaking down the text into individual words or tokens. This step prepares the text for further processing.
- **HashingTF (Hashing Term Frequency):** HashingTF is applied to the tokenized text, which converts it into a term frequency representation. This process involves mapping each token to a numerical index in a fixed-size feature vector, using a hashing function.
- **TF-IDF (Term Frequency-Inverse Document Frequency):** IDF (Inverse Document Frequency) is applied to the output of HashingTF to compute the TF-IDF matrix. This matrix represents the importance of each term in a document relative to its frequency in the entire corpus, effectively weighting down common terms and highlighting rare ones.

B. Model

We have used Logistic Regression and a Naive Bayes model

- **Logistic Regression** In our example, the logistic regression model, which is the most popular, predicts a trinary outcome, categorizing reviews as neutral, negative, or

positive. Additionally, the distribution of these labels is displayed as percentages.

positive_probability	negative_probability	neutral_probability
0.4632494760455983	0.49140876763222174	0.045341756322179884
0.9985120742772488	8.01038668506066E-4	6.868870542451442E-4
0.9800487691010156	0.00793182838856413	0.012019402510420305
0.9651296145856033	0.010669991584460855	0.024200393829935912
0.3270631400133382	0.5816071591123431	0.09132970087431877
0.9207561430557185	0.051428968763490894	0.027814888180790667
0.8596314633495219	0.07459092978742556	0.06577760686305253
0.9680995640514384	0.020040618810776772	0.01185981713778494
0.9469561924895658	0.03763666314432873	0.015407144366105437
0.919721027709641	0.03696316585524219	0.04331580643511671
0.5735606535163694	0.36692067937318795	0.05951866711044263
0.7526093147054281	0.1648094706192822	0.08258121467528964
0.99310462376342	4.4690964136418426E-4	0.006448466595215854
0.9967142236602873	0.0014788203061259103	0.001806956033586876
0.9935494149137075	6.98064211790885E-4	0.005752520874501644
0.9918613869690416	0.0012730958666668137	0.006865517164291671
0.9836321525858835	0.00406936412032309	0.012298483293793346
0.9680350753907859	0.012977217737549769	0.01898770687166436
0.647158914638519	0.22004468265348975	0.13279640270799112
0.9923993592104295	0.002998322922795411	0.004602317866775138

Fig. 3. Example output of model.

- **Naive Bayes** A technique called Naive Bayes is based on the Bayes Theorem.

V. EXPERIMENT AND EVALUATE

Table 1 shows the of models Logistic Regression and Naïve Bayes using different evaluation metrics

TABLE I
PERFORMANCE METRICS

Models	Accuracy	Precision	Recall	F1-Score
Logistic Regression	87.61	86.48	88.21	87.23
Naive Bayes	84.92	81.27	83.92	82.67

We use React framework to build a web application. Figure 4 shows the web application interface of the system. User's comment data is predicted and updated continuously after a short period (typically 10 to 20 seconds).

Sentiment Analysis models integrated inside Spark Structured Streaming will receive streaming data of Youtube's comments from Kafka and predict labels for these comments. Prediction results will be pushed to the dashboard. In addition, the ten latest user comments and predictable labels are also visible on the application.

VI. CONCLUSION

The integration of Kafka and PySpark in real-time sentiment analysis of YouTube comments presents a powerful solution for efficiently processing and analyzing vast amounts of user-generated content. Leveraging Kafka's robust message streaming capabilities and PySpark's distributed computing prowess, this approach enables near-instantaneous sentiment analysis, allowing content creators and platform administrators to gain valuable insights into audience reactions. The seamless combination of these technologies enhances scalability, real-time responsiveness, and the ability to adapt to dynamic

Content	Sentiment	Positive (%)	Negative (%)	Neutral (%)	Compound
oh could I have	positive	54%	0%	47%	32%
Instagram is a trend 6	neutral	0%	0%	100%	0%
يايه اويون باكرين	positive	39%	0%	61%	38%
what's can study like her	positive	51%	0%	49%	64%
...	neutral	0%	0%	100%	0%
تطير حير	neutral	0%	0%	100%	0%
XXXXXXXXXXXXXXXXXXXX	neutral	0%	0%	100%	0%
@leo Chang Eye and to to as well Soldier	positive	10%	0%	81%	27%
yeah an age (Lara) ?	neutral	0%	0%	100%	0%
(head =) (ah)	neutral	0%	0%	100%	0%

Fig. 4. Real-time Sentiment Analysis.

changes in user sentiment. By harnessing the synergy of Kafka and PySpark, this solution offers an effective means to monitor and understand the sentiments expressed in YouTube comments, facilitating informed decision-making and content optimization for creators and platform administrators alike. The Fine Food dataset from Amazon is used in this article's of analysis in a large data set. As part of using Spark MLlib, like Naïve Bayes and Logistic were applied to analyze the feelings in enormous datasets. Apache Spark MLlib is used for huge datasets. 0.3 reviews are used to evaluate the models, while 0.7 reviews are utilized for training the models for the experiment. The Data was explored and analyzed using a few simple processes. The work in the platform the same machine learning algorithm without MapReduce. Continuing our research, we will enhance the interface and incorporate statistical analysis of the number of positive, negative, and neutral comments. We will develop subcategories of negative comments, such as those with an aggressive tone and those that are more subtle. While our current dataset is suitable for analyzing sentiment towards items, we will attempt to integrate additional datasets to better identify individual negative sentiments.

REFERENCES

- [1] S. Shayaa, N. I. Jaafar, S. Bahri, A. Sulaiman, P. S. Wai, Y. W. Chung, "Sentiment Analysis of Big Data: Methods, Applications, and Open Challenges," IEEE Access, vol. 6, pp. 37807-37827, June 28, 2018. Electronic ISSN: 2169-3536.
- [2] M. U. Haque, L. H. Iwaya, M. A. Babar, "Challenges in Docker Development: A Large-scale Study Using Stack Overflow," in Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), October 2020, Article No.: 7, pp. 1-11. DOI: 10.1145/3382494.3410693. Published: October 23, 2020.
- [3] K. Fahd, S. Parvin, A. de Souza-Daw, "A Framework for Real-time Sentiment Analysis of Big Data Generated by Social Media Platforms," in 2021 31st International Telecommunication Networks and Applications Conference (ITNAC), Sydney, Australia, November 24-26, 2021. DOI: 10.1109/ITNAC53136.2021.9652148. Publisher: IEEE. Date Added to IEEE Xplore: December 27, 2021.
- [4] KA. Bhasin, S. Das, "Twitter Sentiment Analysis using Machine Learning and Hadoop: A Comparative Study," in 2021 2nd International Conference on Secure Cyber Computing and Communications (ICSCCC), Jalandhar, India, May 21-23, 2021. DOI: 10.1109/ICSCCC51823.2021.9478077. Publisher: IEEE. Date Added to IEEE Xplore: July 13, 2021.